Project Usability Testing

In this assignment, students have the opportunity to demonstrate proficiency in the following SLOs:

- VV14: Create and execute a User Test Script that effectively elicits feedback from a user.
- VV15. Execute a Usability Test Script that effectively elicits feedback from a user, using the think-aloud protocol.
- VV16. Report the results of a Usability Test Script, including recommendations for improving usability.
- CO6: Communicate with stakeholders effectively, in both technical and non-technical ways.
- RE3. Incorporate feedback from users in a small software project.

During each of the 3 sprints for this phase, students have the opportunity to demonstrate proficiency in the following SLOs:

- T1. Follow effective team communication practices.
- T4. Collaborate effectively in cooperative development/programming.
- TE3. Identify configuration items and use a source code control tool in a small team-based project.
- SC1. Write appropriate unit tests for a small component (several functions, a single type, etc.).
- SC2. Write appropriate interface and (if needed) implementation comments for a small component.
- SC4. Select and use a defined coding standard in a small software project.
- SC5. Write implementation documentation for tricky and non-obvious pieces of code.
- SC6. Use test-driven development in coding.
- VV12. Automate testing in a small software project.
- VV13. Use a defect tracking tool to manage software defects in a small software project.

For this deliverable, your team should run a usability test of your prototype. Your subjects will include your team stakeholders and others, perhaps including students from CS-108. Use the thinking-aloud protocol and plan for a 15–20 minute test.

- **Usability Test Script** — Write a usability test script that covers the key user stories supported by your prototype. Write out a detailed script that includes the following elements:
    - *Introduction* — Prepare your test subjects by briefly explaining the vision of your system, the goals that they are to adopt during the test, and, if necessary, the key elements of the domain of application they need to know.
    - *Background Questions* — Answer any questions your subjects have at this point and ask them for information that is relevant to your test. Encourage them here to talk aloud while they are using your system.
    - *The Test* — Specify the user tasks required for the test. Base these tasks on key user stories and don't include procedural details, e.g., say "Find out how many monopoly game players there are." rather than "Go to the player list page and count the number of player rows."
    - *Thank You* — A (heart-felt) thank-you for their time and effort.

    - As a model, you can use our Krug-inspired test script for a test of the CS department website, see csWebsiteTestScript.pdf (DOCX version) (cf. Steve Krug's "Usability test script" for mobile apps and other downloadable material referenced on his *Rocket Surgery Made Easy* downloads page). You'll pilot a draft of this test script sometime before Thanksgiving break begins. Push your final test script to your project repo.

- **Pilot Studies** — Run a pilot usability study with your professor as the subject, and then with your team stakeholders, both before Thanksgiving. Use those experiences to iteratively revise your test script as appropriate.

- **Test Management** — Run your team usability test.
    - *Test Facilitation* — Your whole team does not need to attend the tests. We suggest having two people there, one to do the talking and the other to do the recording. During the test, you may answer general questions, but don't tell your subjects how to do things. Step in only if they really get stuck. If they stop talking, ask them questions (e.g., "What are you trying to do now?", "Is the system clear to you so far?").
    - *Test Subjects* — Treat your subjects well. You may run them in teams rather than as individuals if that makes sense for your application.
    - *Test Venue* — Run your subjects in as realistic a venue as possible. If your system is an outside system, then go outside with your subjects. Try to run the tests in as sequestered a way as possible so that each subject (or subject team) doesn't get distracted by the work of other subjects.
    - *Test Incidents* — Report any incidents (i.e., observed potential failures) that happen during the test or significant suggested improvements as tickets/tasks in your project. Use the GitHub issue tracker.
    - *Test Report* — Write up a 1-page report describing the user tasks you tested, what you discovered, and what you plan to do about it (see an example report here: CalvinFinds). Notes:
        - Don't mention names in this report.
        - Push it to your project repo and announce/distribute it to your subjects, your teammates, your stakeholders, and us.
        - Be sure to use BCC for email to keep the subjects from finding out about each other.
    - *Participation Report* — Email your professor (only) a list of the test subjects who showed up for your test, regardless of whether they completed all the tasks.

- **Grading** — Neither you nor your subjects will be graded on how "well" the subjects perform the test tasks. Participation and valid feedback are the only requirements.
    - *Subject Grading* — Your subjects will get full credit for their participation if they show up and do at least part of the test. CS 262 students will receive credit for their participation as subjects as part.

Execute your usability tests during the week following Thanksgiving and upload your report to your project website by the end of the day Friday of that week.